

Fall Term 2024

Title:	Assignment 3	Due Date:	27-12-2024 23:59:59
Instructor:	Muhammad Yasin Nasir	Assig Date:	17-12-2024
Book:	Assembly Language Programming Organization of the IBM PC by Ytha Yu		
Subject:	Computer Organization & Assembly Language		
Syllabus:	Shifting, Rotation, Stack, Procedures, Multiplication, Division, String		
Name:		Roll No.	

Instructions:

01. Questions are SELF EXPLANATORY. Understanding the Question is part of Solution.
02. Error in Question will be advantageous to Student.
03. Read the Questions carefully before attempting.
04. Attempt All Questions in a Precise Fashion.
05. Manage Your Time. Therefore, there will be no extension in the deadline.
06. Plagiarism may lead to F grade, so be honest.

Submission Method:

01. Assignment must be submitted individually.
02. **Take print out of the given assignment. Use A4 size paper** (210 × 297 millimeters or 8.27 × 11.69 inches) to print it out. **Hand-written** assignment will be accepted as hard copy. Attempt the questions on blank A4 paper. Attach the outputs of the screen in printed form.
03. Use any scanner tool to convert it to pdf. Text must have clear visibility after scanning. Blurred pdfs will be rejected.
04. Upload the solved assignment (soft copy) at [university portal](#) before the deadline along with asm files.
05. Submit the assignment (hard from) during the first half hour of the subsequent lecture.

Guidelines:

01. Use your official name on the SAP with first letter of each string as capital alphabets and others as small letters.
02. Use your complete SAP ID.
03. Apply **validation** to each input string/number/character.
04. Use the following pattern for file name: sapID_Q1_A2.asm
05. Submit only one file of asm program.
So, you need to assemble all parts of the question in one file.

Q 1: Write an assembly language program for the given statement.

Part a: OUTDEC: Design a procedure that can display decimal values with multiple digits.

Part b: INDEC: Design a procedure that can input decimal values with multiple digits.

Part c: OUTHEX: Design a procedure that can display HEXADECIMAL values with FOUR digits.

Part d: INDHEX: Design a procedure that can input HEXADECIMAL values with FOUR digits.

Part e: OUTBIN: Design a procedure that can display BINARY values with SIXTEEN BITS.

Part f: INBIN: Design a procedure that can input BINARY values with SIXTEEN BITS.

Part g: Write a main procedure to call all the above procedures in appropriate way.

Q 2: Write an assembly language program for the given statements. Use separate procedure for each part.

Procedure name is mentioned at the start of each part.

Part a: INPUT_NAME: Input a name as string terminated by \$ and save it in memory (Use function 1 of INT 21H, loop to implement the operation, and register indirect addressing mode to address the memory).

Part b: CASE_CONVERSION: Convert the saved string's case (capital $\rightleftharpoons$ small) and save the string with case conversion. (Use logical operation with loop, use based addressing mode to address the memory locations).

Part c: VOWELS: Use **part a**, calculate the number of vowels in the string. Use OUTDEC to display the value.

Part d: CONSONANTS: Use **part a**, calculate the number of consonants in the string. Use OUTDEC to display the value.

Part e: BINARY_CONVERSION: Use **part a**, convert the saved ASCII values of whole string to binary values and save the binary characters in the memory. Use OUTBIN to display the whole string.

Part f: HEXADECIMAL_CONVERSION: Use **part a**, convert the saved ASCII values of whole string to hexadecimal values and save the hexadecimal characters in the memory. Use OUTHEX to display the whole string.

Part g: ONES_BITS: Use **part e**, find the numbers of ones' bits in the whole string. (Use indexed addressing mode to address the memory). Use OUTDEC as well.

Part h: ZEROS_BITS: Use **part e**, find the number of zeros' bits in the whole string. (Use based addressing mode to address the memory). Use OUTDEC to display the count of zero bits.

Part i: REVERSE_STRING: Use **part a**, reverse the string and save it in the memory. (Use based and indexed addressing mode). Display the reversed string.

Part j: WITHOUT_VOWELS: Use **part a & c**, remove the vowels from the string and save it in the memory. Display the string without vowels.

Part k: WITHOUT_CONSONANTS: Use **part a & d**, remove the consonants from the string and save it in the memory. Display the string without consonants.

Part l: PRINTING: Print all the strings in the memory separated by new line (NOTE: ignore if string has already been displayed). (Using function 9 of INT21H).